

ARCHITECTURAL MEMORY

Microscope Memory · Public Demo Release

v0.1.0-public-demo · Máté Róbert · Budapest · 2026-07-01

“Memory is not a static vault.
Memory is a living structure.”

269

MEMORY TESTS

16

HOOK TESTS

285

TOTAL PASSING

6

LIFECYCLE HOOKS

0

WRITE TOOLS

01. TEMPORAL EVOLUTION

The public demo release follows a temporal evolution model. Each stage represents a phase in the hardening and verification process, from a fresh safe index to a locked public demo configuration.

T0	Fresh safe index. No private dataset. Read-only runtime.
T100	Safe MCP tools respond: memory.status, memory.search, memory.context.
T1000	Context emerges from existing memory structure. No writes. No mutation.
T5000	Public demo locked. 285 tests passing. Secret filtering active.

02. SAFETY ARCHITECTURE

The public demo runs in a read-only, safety-hardened configuration. No memory writes are permitted. No private dataset paths are configured. All mutation tools are disabled at the MCP layer.

Layer	Setting	Description
read_only	true	No memory writes from hooks
write_enabled	false	after_response hook disabled
Private dataset	—	No private paths configured
Secret filtering	active	Passwords, tokens, API keys filtered
Logging	stderr	Never leaks to stdout

03.

HOOK SYSTEM – LIFECYCLE INTEGRATION

The Hook System is a thin orchestration layer around the existing Microscope Memory API. It allows LLM agents to trigger memory actions automatically at key lifecycle points.

Hook	Event	Function
on_session_start	Session init	Load identity, memory contract, constraints
before_tool_call	Pre-tool	Retrieve task memory, check constraints
	Post-tool	
after_tool_call	ol	Summarize results, prepare memory candidates
on_error	Error	Store error traces, associate with tasks
	Post-response	
after_response	sponse	Extract durable memory (disabled in public mode)

04.

MCP INTEGRATION

The MCP server exposes Microscope Memory through the Model Context Protocol. In public demo mode, only read-only tools are available:

- memory_status – Index status, block count, depths
- memory_recall – Natural language memory search
- memory_find – Text search across all blocks
- memory_look – Spatial coordinate lookup
- memory_auto_context – Session context snapshot
- memory_timeline – Chronological memory view

All write tools (memory_store, memory_build, memory_rebuild, memory_consolidate, memory_dream) are disabled. The MCP server logs exclusively to stderr, never stdout.

05.

RELEASE VERIFICATION

The release checklist in docs/RELEASE_PUBLIC_DEMO.md defines ten verification steps. Each step must pass before the release is considered ready:

- ✓ Full test suite: 285 tests passing
- ✓ Public demo config: read_only=true, write_enabled=false

- ✓ No private dataset paths in configuration
 - ✓ MCP tools/list responds with correct tool set
 - ✓ Write tools are not exposed in tools/list
 - ✓ Secret filtering active on all inputs
 - ✓ Release binary builds cleanly
 - ✓ Version tag v0.1.0-public-demo set
-

06.

ARCHITECTURE

The system follows a layered architecture where the Rust core is the source of truth. All adapters (MCP, WASM, CLI) are thin wrappers around the same core logic.

```
microscope-core (Rust, binary, mmap, sub-microsecond)
+-- microscope-hooks (lifecycle orchestration)
+-- microscope-mcp (MCP server over stdio)
+-- microscope-wasm (browser target)
+-- microscope-cli (command-line interface)
```

The model is only the motor. Hooks are the nervous system. Microscope Memory is the memory.

07.

REFERENCES

01. Microscope Memory — github.com/silentnoisehun/microscope-memory
02. HOPE-Architect Style — Monochrome-Architect PDF generation guide
03. Model Context Protocol — spec.modelcontextprotocol.io
04. Bio-Binaries — HOPE-WP-2026 // BIO-BINARIES · V2.1 · RUST 2021